

Week 6: MC & IoT Lab

1. Check Two Numbers Equal Using BEQ

```
PRESERVE8
THUMB
AREA |.text|, CODE, READONLY
EXPORT __main

__main

    MOV     R0, #25        ; First number
    MOV     R1, #25        ; Second number

    CMP     R0, R1         ; Compare R0 and R1

    BEQ     EQUAL          ; Branch if equal
    BNE     NOTEQUAL       ; Branch if not equal

EQUAL
    MOV     R2, #1         ; Store 1 if equal
    B       STOP

NOTEQUAL
    MOV     R2, #0         ; Store 0 if not equal

STOP
    B       STOP

END
```

2. Demonstrate ADDS Updating APSR Flags

```
PRESERVE8
THUMB
AREA |.text|, CODE, READONLY
EXPORT __main

__main

    MOV     R0, #10
    MOV     R1, #20

    ADDS    R2, R0, R1      ; R2 = 30 (Updates APSR Flags)

    CMP     R2, #30

    BEQ     RESULT

    MOV     R3, #0
    B       STOP

RESULT
    MOV     R3, #1

STOP
    B       STOP
```

END

3. Find Greater Number Using Conditional Branch

```
PRESERVE8
THUMB
AREA |.text|, CODE, READONLY
EXPORT __main

__main

    MOV     R0, #15
    MOV     R1, #35

    CMP     R0, R1

    BGT     R0_GREATER
    BLT     R1_GREATER

R0_GREATER
    MOV     R2, R0
    B       STOP

R1_GREATER
    MOV     R2, R1

STOP
    B       STOP

END
```

4. Add Two Word (32-bit) Numbers using DCD

```
PRESERVE8
THUMB

AREA |.data|, DATA, READWRITE

WORD1    DCD    0x12345678
WORD2    DCD    0x87654321
RESULT   DCD    0

AREA |.text|, CODE, READONLY
EXPORT __main

__main

    LDR     R0, =WORD1
    LDR     R1, [R0]

    LDR     R0, =WORD2
    LDR     R2, [R0]

    ADD     R3, R1, R2

    LDR     R0, =RESULT
    STR     R3, [R0]
```

```

STOP
    B STOP

END

```

Register Value

```

R1      0x12345678
R2      0x87654321
R3      0x99999999
RESULT  0x99999999

```

5. Add Two Halfword Numbers using DCW

```

        PRESERVE8
        THUMB

        AREA |.data|, DATA, READWRITE

HALF1   DCW      0x1234
HALF2   DCW      0x5678
RESULT  DCW      0

        AREA |.text|, CODE, READONLY
        EXPORT __main

__main

        LDR      R0, =HALF1
        LDRH     R1, [R0]

        LDR      R0, =HALF2
        LDRH     R2, [R0]

        ADD      R3, R1, R2

        LDR      R0, =RESULT
        STRH     R3, [R0]

STOP
    B STOP

END

```

Register Value

```

R1      0x1234
R2      0x5678
R3      0x68AC
RESULT  0x68AC

```

6. Add Two Byte Numbers using DCB

```

        PRESERVE8
        THUMB

        AREA |.data|, DATA, READWRITE

BYTE1   DCB      0x12
BYTE2   DCB      0x34
RESULT  DCB      0

        AREA |.text|, CODE, READONLY
        EXPORT __main

__main

        LDR      R0, =BYTE1
        LDRB     R1, [R0]

        LDR      R0, =BYTE2
        LDRB     R2, [R0]

        ADD      R3, R1, R2

        LDR      R0, =RESULT
        STRB     R3, [R0]

STOP

        B STOP

        END

```

Register Value

R1	0x12
R2	0x34
R3	0x46
RESULT	0x46

7. Find string length

```

        PRESERVE8
        THUMB

        AREA |.data|, DATA, READWRITE

TXT      DCB      "HELLO", 0
LEN      DCD      0

        AREA |.text|, CODE, READONLY
        EXPORT __main

__main

        LDR      R0, =TXT
        MOV      R1, #0

```

```

LOOP
    LDRB    R2, [R0], #1
    CMP     R2, #0
    BEQ     DONE

    ADD     R1, R1, #1
    B LOOP

DONE
    LDR     R0, =LEN
    STR     R1, [R0]

STOP
    B STOP

END

```

Register Value

```

String  HELLO
Length  5
R1      5
LEN     5

```

8. Write an ARM assembly program to move data between registers and immediate values.

```

PRESERVE8
THUMB

AREA |.text|, CODE, READONLY
EXPORT __main

__main

    MOV     R0, #25        ; Load immediate value
    MOV     R1, R0         ; Copy register

    MOVS    R2, R1         ; Copy and update flags
    ADDS    R3, R2, #0     ; Copy using ADDS

STOP
    B STOP

END

```

Register Value

```

R0      25
R1      25
R2      25
R3      25

```

9. Read special registers using MRS instruction

```

PRESERVE8
THUMB

```

```

        AREA |.text|, CODE, READONLY
        EXPORT __main

__main

        MRS      R0, CONTROL
        MRS      R1, PRIMASK
        MRS      R2, xPSR

STOP

        B STOP

END

```

Register Description

R0	CONTROL Register value
R1	PRIMASK value
R2	xPSR value

10. Load and store data from memory

```

        PRESERVE8
        THUMB

        AREA |.data|, DATA, READWRITE

NUM1    DCD      10
RESULT  DCD      0

        AREA |.text|, CODE, READONLY
        EXPORT __main

__main

        LDR      R0, =NUM1
        LDR      R1, [R0]

        ADD      R2, R1, #5

        LDR      R0, =RESULT
        STR      R2, [R0]

STOP

        B STOP

END

```

Register Value

R1	10
R2	15
RESULT	15

11. Load byte and halfword data

```

        PRESERVE8

```

```

        THUMB

        AREA |.data|, DATA, READWRITE

BYTE1   DCB    0x12
HALF1   DCW    0x1234

        AREA |.text|, CODE, READONLY
        EXPORT __main

__main

        LDR     R0, =BYTE1
        LDRB    R1, [R0]

        LDR     R0, =HALF1
        LDRH    R2, [R0]

STOP
        B STOP

        END

```

Register Value

R1	0x12
R2	0x1234

12. Store word, halfword and byte

```

        PRESERVE8
        THUMB

        AREA |.data|, DATA, READWRITE

MEM      DCD 0,0,0

        AREA |.text|, CODE, READONLY
        EXPORT __main

__main

        MOV     R1, #100
        MOV     R2, #50
        MOV     R3, #25

        LDR     R0, =MEM

        STR     R1, [R0]
        STRH    R2, [R0,#4]
        STRB    R3, [R0,#6]

STOP
        B STOP

        END

```

Memory Location Value

MEM	100
-----	-----

Memory Location Value

MEM+4	50
MEM+6	25

13. Load multiple registers

```
PRESERVE8
THUMB

AREA |.data|, DATA, READWRITE

DATA    DCD 10,20,30,40

AREA |.text|, CODE, READONLY
EXPORT __main

__main

    LDR    R0, =DATA

    LDM    R0, {R1-R4}

STOP

    B STOP

END
```

Register Value

R1	10
R2	20
R3	30
R4	40

14. Store multiple registers

```
PRESERVE8
THUMB

AREA |.data|, DATA, READWRITE

MEM      DCD 0,0,0,0

AREA |.text|, CODE, READONLY
EXPORT __main

__main

    MOV R1,#11
    MOV R2,#22
    MOV R3,#33
    MOV R4,#44

    LDR R0, =MEM

    STM R0, {R1-R4}
```

STOP

B STOP

END

Memory Value

MEM 11

MEM+4 22

MEM+8 33

MEM+12 44
